

AI-Based Crop Disease Prediction System

Complete Notebook Code — MyAgri / IBM PBEL

Copy each cell into one Jupyter notebook in the order shown. Save the notebook as 01_complete_crop_disease_pipeline.ipynb.

Cell 1 — Introduction

```
# AI-Based Crop Disease Prediction System
## MyAgri | IBM PBEL AI Solution
```

This project predicts crop diseases from leaf images using deep learning and EfficientNet-B0.

Objectives:

- Validate and explore PlantVillage
- Detect corrupted and duplicate images
- Analyze class imbalance
- Create 70/15/15 stratified splits
- Apply preprocessing and augmentation
- Train EfficientNet-B0
- Evaluate accuracy, precision, recall and Macro F1
- Generate a confusion matrix
- Save the best model

Dataset: PlantVillage, approximately 54,000 labeled plant-leaf images.

Important: use this for research and preliminary screening; predictions are not definitive agricultural diagnoses.

Cell 2 — Imports

```
import random, hashlib
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image, UnidentifiedImageError

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models
from torchvision.models import EfficientNet_B0_Weights

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix, ConfusionMatrixDisplay
)

print("Libraries imported successfully.")
```

Cell 3 — Configuration

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

DATASET_PATH = Path("../data/raw/PlantVillage")
PROCESSED_PATH = Path("../data/processed")
REPORTS_PATH = Path("../reports")
MODELS_PATH = Path("../models")

for p in [PROCESSED_PATH, REPORTS_PATH, MODELS_PATH]:
    p.mkdir(parents=True, exist_ok=True)

IMAGE_SIZE = 224
BATCH_SIZE = 32
EPOCHS = 10
LEARNING_RATE = 0.001
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Device:", DEVICE)
print("Dataset:", DATASET_PATH.resolve())
```

Cell 4 — Discover Images

```
EXTENSIONS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

image_paths = [
    p for p in DATASET_PATH.rglob("*")
    if p.is_file() and p.suffix.lower() in EXTENSIONS
]
print(f"Total images found: {len(image_paths):,}")
```

Cell 5 — Metadata

```
records = []

for path in image_paths:
    class_name = path.parent.name
    if "___" in class_name:
        crop, disease = class_name.split("___", 1)
    else:
        crop, disease = "Unknown", class_name

    records.append({
        "image_path": str(path),
        "class": class_name,
        "crop": crop,
        "disease": disease
    })

df = pd.DataFrame(records)
print("Dataset shape:", df.shape)
df.head()
```

Cell 6 — Dataset Statistics

```
print("Total Images:", len(df))
print("Total Classes:", df["class"].nunique())
print("Total Crops:", df["crop"].nunique())
print(df["crop"].value_counts())
```

Cell 7 — Corrupted Images

```
valid_records = []
corrupted_images = []

for _, row in df.iterrows():
    try:
        with Image.open(row["image_path"]) as image:
            image.verify()
            valid_records.append(row)
    except (UnidentifiedImageError, OSError, ValueError):
        corrupted_images.append(row["image_path"])

clean_df = pd.DataFrame(valid_records).reset_index(drop=True)

print("Original:", len(df))
print("Valid:", len(clean_df))
print("Corrupted:", len(corrupted_images))
```

Cell 8 — Class Distribution

```
class_distribution = clean_df["class"].value_counts().sort_values(ascending=False)
print(class_distribution)

plt.figure(figsize=(14, 8))
class_distribution.sort_values().plot(kind="barh")
plt.xlabel("Number of Images")
plt.ylabel("Disease Class")
plt.title("PlantVillage Class Distribution")
plt.tight_layout()
plt.savefig(REPORTS_PATH / "class_distribution.png", dpi=300)
plt.show()
```

Cell 9 — Imbalance

```
maximum = class_distribution.max()
minimum = class_distribution.min()
imbalance_ratio = maximum / minimum

print("Largest class:", maximum)
print("Smallest class:", minimum)
print(f"Imbalance ratio: {imbalance_ratio:.2f}")
```

Cell 10 — Samples

```
sample_paths = random.sample(
    list(clean_df["image_path"]), min(16, len(clean_df))
)

plt.figure(figsize=(14, 12))

for i, path in enumerate(sample_paths):
    image = Image.open(path).convert("RGB")
    plt.subplot(4, 4, i + 1)
    plt.imshow(image)
    plt.title(Path(path).parent.name, fontsize=8)
    plt.axis("off")
```

```
plt.suptitle("Random Plant Disease Samples")
plt.tight_layout()
plt.savefig(REPORTS_PATH / "sample_images.png", dpi=300)
plt.show()
```

Cell 11 — Image Dimensions

```
dimensions = []

for path in clean_df["image_path"]:
    try:
        with Image.open(path) as image:
            width, height = image.size
            dimensions.append({
                "width": width,
                "height": height,
                "aspect_ratio": width / height
            })
    except Exception:
        pass

dimensions_df = pd.DataFrame(dimensions)
print(dimensions_df.describe())
```

Cell 12 — Duplicate Detection

```
def md5_hash(path):
    h = hashlib.md5()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b""):
            h.update(chunk)
    return h.hexdigest()

clean_df["hash"] = [
    md5_hash(path) for path in clean_df["image_path"]
]

duplicates = clean_df[clean_df.duplicated("hash", keep=False)]
print("Images involved in exact duplicates:", len(duplicates))

clean_df = clean_df.drop_duplicates("hash").reset_index(drop=True)
```

Cell 13 — 70/15/15 Split

```
train_df, temp_df = train_test_split(
    clean_df, test_size=0.30, random_state=SEED,
    stratify=clean_df["class"]
)

validation_df, test_df = train_test_split(
    temp_df, test_size=0.50, random_state=SEED,
    stratify=temp_df["class"]
)

print("Training:", len(train_df))
print("Validation:", len(validation_df))
print("Testing:", len(test_df))

total = len(clean_df)
print(f"Train: {len(train_df)/total:.2%}")
print(f"Validation: {len(validation_df)/total:.2%}")
print(f"Test: {len(test_df)/total:.2%}")
```

Cell 14 — Save Splits

```
train_df.to_csv(PROCESSED_PATH / "train.csv", index=False)
validation_df.to_csv(PROCESSED_PATH / "validation.csv", index=False)
test_df.to_csv(PROCESSED_PATH / "test.csv", index=False)
print("Splits saved.")
```

Cell 15 — Transforms

```
train_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                          [0.229, 0.224, 0.225])
])

validation_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                          [0.229, 0.224, 0.225])
])
```

Cell 16 — Dataset Class

```
class_names = sorted(train_df["class"].unique())
class_to_idx = {name: i for i, name in enumerate(class_names)}

class CropDiseaseDataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.dataframe = dataframe.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.dataframe)

    def __getitem__(self, index):
        row = self.dataframe.iloc[index]
        image = Image.open(row["image_path"]).convert("RGB")
        label = class_to_idx[row["class"]]
        if self.transform:
            image = self.transform(image)
        return image, label

print("Classes:", len(class_names))
```

Cell 17 — DataLoaders

```
train_dataset = CropDiseaseDataset(train_df, train_transform)
validation_dataset = CropDiseaseDataset(validation_df, validation_transform)
test_dataset = CropDiseaseDataset(test_df, validation_transform)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
validation_loader = DataLoader(validation_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)

print("DataLoaders ready.")
```

Cell 18 — EfficientNet-B0

```
model = models.efficientnet_b0(
    weights=EfficientNet_B0_Weights.DEFAULT
)

features = model.classifier[1].in_features
model.classifier[1] = nn.Linear(features, len(class_names))
model = model.to(DEVICE)

criterion = nn.CrossEntropyLoss()

optimizer = optim.AdamW(
    model.parameters(),
    lr=LEARNING_RATE,
    weight_decay=1e-4
)

scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="max", patience=2, factor=0.5
)

print(model.classifier)
```

Cell 19 — Accuracy Function

```
def calculate_accuracy(model, loader):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(DEVICE)
            labels = labels.to(DEVICE)
            outputs = model(images)
            predictions = outputs.argmax(dim=1)
            correct += (predictions == labels).sum().item()
            total += labels.size(0)

    return correct / total if total else 0.0
```

Cell 20 — Training

```
history = []
best_accuracy = 0.0

for epoch in range(EPOCHS):
    model.train()
    running_loss = 0.0

    for images, labels in train_loader:
        images = images.to(DEVICE)
        labels = labels.to(DEVICE)
```

```

optimizer.zero_grad()
outputs = model(images)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()

running_loss += loss.item()

validation_accuracy = calculate_accuracy(model, validation_loader)
scheduler.step(validation_accuracy)

epoch_loss = running_loss / len(train_loader)

history.append({
    "epoch": epoch + 1,
    "loss": epoch_loss,
    "validation_accuracy": validation_accuracy
})

print(
    f"Epoch {epoch+1}/{EPOCHS} | "
    f"Loss: {epoch_loss:.4f} | "
    f"Val Accuracy: {validation_accuracy:.4f}"
)

if validation_accuracy > best_accuracy:
    best_accuracy = validation_accuracy
    torch.save(
        {
            "model_state_dict": model.state_dict(),
            "class_names": class_names
        },
        MODELS_PATH / "best_efficientnet_b0.pth"
    )

print("Best validation accuracy:", best_accuracy)

```

Cell 21 — Training Graph

```

history_df = pd.DataFrame(history)

fig, ax1 = plt.subplots(figsize=(10, 6))
ax1.plot(history_df["epoch"], history_df["loss"], label="Training Loss")
ax1.set_xlabel("Epoch")
ax1.set_ylabel("Loss")

ax2 = ax1.twinx()
ax2.plot(history_df["epoch"], history_df["validation_accuracy"],
         label="Validation Accuracy")
ax2.set_ylabel("Validation Accuracy")

plt.title("Training Performance")
plt.tight_layout()
plt.savefig(REPORTS_PATH / "training_performance.png", dpi=300)
plt.show()

```

Cell 22 — Test Predictions

```

checkpoint = torch.load(
    MODELS_PATH / "best_efficientnet_b0.pth",
    map_location=DEVICE
)

model.load_state_dict(checkpoint["model_state_dict"])
model.eval()

y_true = []
y_pred = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(DEVICE)
        outputs = model(images)
        predictions = outputs.argmax(dim=1).cpu().numpy()

        y_pred.extend(predictions)
        y_true.extend(labels.numpy())

```

Cell 23 — Final Metrics

```

accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average="macro", zero_division=0)
recall = recall_score(y_true, y_pred, average="macro", zero_division=0)
f1 = f1_score(y_true, y_pred, average="macro", zero_division=0)

print("FINAL TEST RESULTS")
print("=" * 40)
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"Macro F1  : {f1:.4f}")

```

Cell 24 — Classification Report

```
report = classification_report(
    y_true, y_pred,
    target_names=class_names,
    zero_division=0
)

print(report)

with open(REPORTS_PATH / "classification_report.txt", "w") as f:
    f.write(report)
```

Cell 25 — Confusion Matrix

```
cm = confusion_matrix(y_true, y_pred)

fig, ax = plt.subplots(figsize=(18, 18))

display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=class_names
)

display.plot(ax=ax, xticks_rotation=90, colorbar=False)

plt.title("Crop Disease Classification")
plt.tight_layout()
plt.savefig(REPORTS_PATH / "confusion_matrix.png",
            dpi=300, bbox_inches="tight")
plt.show()
```

Cell 26 — Save Results

```
results = pd.DataFrame([
    {
        "model": "EfficientNet-B0",
        "dataset": "PlantVillage",
        "accuracy": accuracy,
        "macro_precision": precision,
        "macro_recall": recall,
        "macro_f1": f1
    }
])

results.to_csv(
    REPORTS_PATH / "model_metrics.csv",
    index=False
)

results
```

Cell 27 — Final Markdown

■ Final Results

The model was trained on the PlantVillage dataset and evaluated on a held-out test set.

The metrics above are generated directly by the executed notebook.

```
## Key Components
- Dataset validation
- Corrupted-image detection
- Exact duplicate detection
- Stratified train/validation/test splitting
- Data augmentation
- EfficientNet-B0 transfer learning
- Test-set evaluation
- Classification report
- Confusion matrix
```

```
## Future Improvements
- Field-condition Indian crop images
- Mobile/edge inference
- Grad-CAM explainability
- Disease severity estimation
- Weather and soil context
- Multilingual farmer interface
- Model comparison
```

Do not claim an accuracy until this notebook has been executed on the real dataset.